Introduction to PySpark Dataframes

PySpark Dataframes:

- Higher-level abstraction on top of Spark's RDD.
- Optimized for structured and tabular data.
- Conceptually the same as a database table.
- Can be used in the same way as a dataframe in Pandas or R.

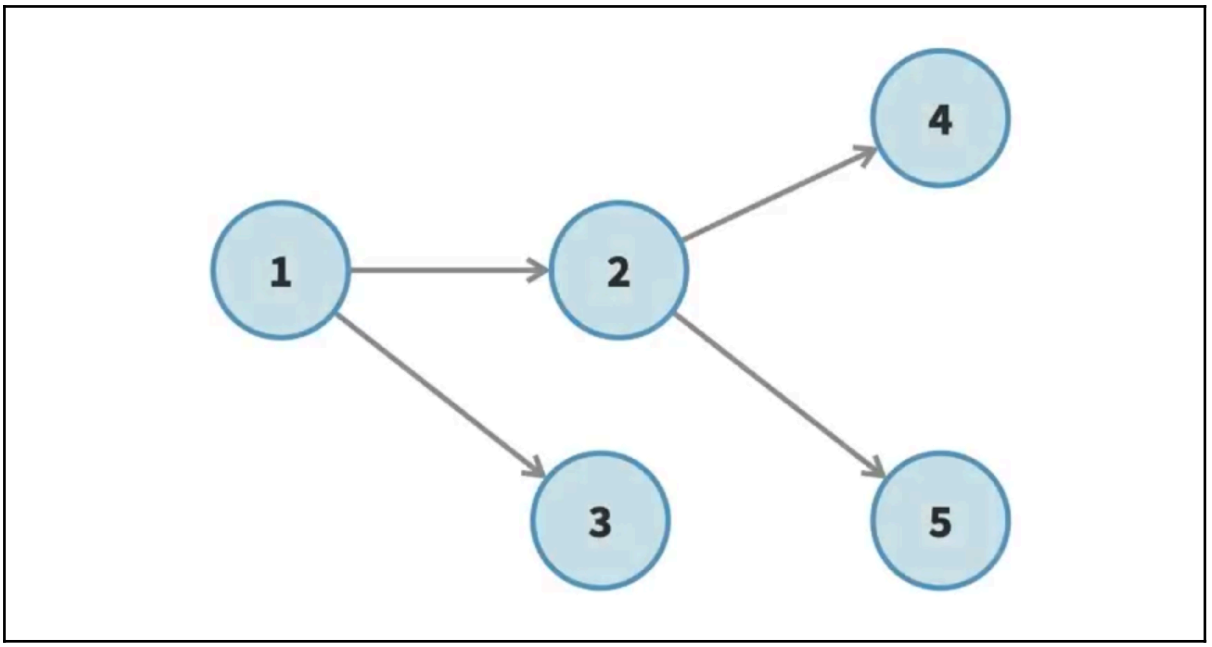
PySpark Dataframes Vs. Pandas Dataframes

PySpark Dataframe	Pandas Dataframes
Designed for large, distributed datasets as they are distributed across multiple machines in a cluster.	Designed for small to medium-size datasets as they are all held in memory on a single machine.
These dataframes are immutable, which means any operation performed on it returns a new dataframe.	These dataframes are mutable, which means that the dataframe itself can be modified with an operation.
Uses lazy evaluation, which means that the operations are queued and optimized for execution.	Uses eager evaluation, meaning the operations are executed as soon as they occur in the code.

Since PySpark uses lazy evaluation, spark doesn't immediately run the code when you apply transformations to it. Instead it builds a DAG (Directed Acyclic Graph) behind the scenes.

Directed Acyclic Graph (DAG)

You can think of the DAG as a roadmap to our operations.



This DAG shows us:

- What data are we starting with?
- What transformations do we want to apply?
- What's the best order to optimise computation?

This DAG is what allows spark to optimize our job before running it. It can figure out the most efficient way to execute geotransformations across the cluster, group related steps together and cut out unnecessary work.

In summary, we can say that while both pandas and pyspark dataframes provide a similar data structure and operations, the way they work under the hood is very different.